

Machine Learning per la Segmentazione

Il *Machine Learning* (ML) rappresenta un insieme di algoritmi che consentono al computer di apprendere da un set di dati e fare predizioni o prendere decisioni, senza essere esplicitamente programmato per un compito specifico.

Nel contesto della segmentazione delle immagini biomediche, gli algoritmi ML vengono utilizzati per **imparare le relazioni tra i *pattern* di intensità e le *texture*** dei pixel/voxel e l'appartenenza di questi a una specifica struttura anatomica (*label*).

Distinzione tra Apprendimento Supervisionato e Non Supervisionato

Gli algoritmi di *Machine Learning* (e i relativi approcci alla segmentazione) si dividono in due categorie principali a seconda del modo in cui elaborano i dati: !Pasted image 20251129201800.png

Apprendimento Supervisionato (*Supervised Learning*) L'apprendimento supervisionato è l'approccio più comune nella segmentazione clinica, specialmente con il *Deep Learning*.

- **Definizione:** L'algoritmo viene addestrato utilizzando un **set di dati di *training* validato e etichettato**.
- **Dati:** Il *training set* è costituito da coppie (**Immagine, Maschera di Riferimento o *Ground Truth***). La maschera (*Ground Truth*) è creata manualmente da esperti e rappresenta l'obiettivo di segmentazione ideale.
- **Obiettivo:** L'algoritmo impara a mappare gli *input* (immagini) agli *output* desiderati (maschere etichettate) **minimizzando una funzione di errore** (o *loss function*) tra la sua predizione e la *Ground Truth*.
- **Esempi:** Segmentazione di organi specifici (es. cuore, rene) o tumori, utilizzando Reti Neurali Convolutionali (CNN).

Principio Generale Nel *supervised learning*, si suppone l'esistenza di un sistema con una certa **funzione di trasferimento** $H()$ che mappa un certo *input* (I) in un certo *output* (O).

- **Esempio Biomedico:** L'*input* I potrebbe essere un'immagine biomedica (es. TAC), e l'*output* O la maschera (*mask*) risultante dal processo di segmentazione.
- **Obiettivo del Training:** Partendo da una coppia di esempio **Input-Output** (I_T, O_T), detta **Training Set** o insieme di addestramento, si vuole determinare $H()$ tale che la differenza, o errore, tra l'*output* predetto $H(I_T)$ e l'*output* atteso O_T sia la più piccola possibile:

$$\text{Errore} = \text{Distanza}(H(I_T), O_T) \rightarrow \min$$

- **Generalizzazione:** L'obiettivo secondario, ma cruciale, è che, dato un nuovo insieme di dati mai visti prima (I_1, O_1), la differenza $H(I_1) - O_1$ sia ancora piccola. Si tratta quindi di **“imparare”** da un esempio una regola generale che funzioni anche con set di dati simili (capacità di **generalizzazione**).
-

Classificazione in Base alla Natura dei Dati di Output A seconda della natura dei dati di *output* che il sistema deve predire, si distinguono tre casi principali:

Classification Learning (Classificazione)

- **Natura dell'Output:** I dati di *output* non hanno una struttura metrica particolare; l'unica cosa che si può determinare è se due elementi dell'insieme dei dati di *output* siano **uguali o meno**.
- **Elementi:** Gli elementi dello spazio di *output* si chiamano **Classi**.
- **Obiettivo:** Assegnare l'input a una delle classi predefinite.
- **Esempio:** Riconoscimento dei caratteri scritti a mano, dove le classi sono tutti i possibili caratteri.

Preference Learning (Ranghi)

- **Natura dell'Output:** Tra due elementi non uguali è possibile determinare quale sia il **migliore** e quale il **peggiore**, ma **non** è possibile graduare la differenza tra i due elementi.
- **Elementi:** Gli elementi dello spazio di *output* sono chiamati **Ranks** (ranghi).
- **Obiettivo:** Stabilire un ordinamento tra gli elementi. Esiste un ordinamento, ma non il concetto di distanza metrica tra di essi.
- **Esempio:** Sistemi di raccomandazione che classificano i prodotti in ordine di preferenza, senza quantificare quanto una preferenza sia maggiore dell'altra.

Function Learning (Regressione)

- **Natura dell'Output:** Lo spazio di *output* è uno **spazio metrico**, e quindi a ogni elemento è assegnato il valore di una funzione che è **continua**.
- **Elementi:** L'output è un valore numerico continuo.
- **Obiettivo:** Predire un valore numerico preciso.
- **Algoritmi Avanzati:** In questo caso, come si vedrà esaminando le reti neurali, l'esistenza di uno spazio metrico permette l'utilizzo dell'algoritmo di **Back-Propagation**, che garantisce il raggiungimento di un massimo locale (ottimizzazione).
- **Esempio:** Predizione dell'età ossea di un paziente (un valore numerico continuo) a partire da un'immagine radiografica.

Apprendimento Non Supervisionato (*Unsupervised Learning*) L'apprendimento non supervisionato viene utilizzato per scoprire strutture nascoste nei dati non etichettati.

- **Definizione:** L'algoritmo non utilizza dati di *training* validati.
- **Obiettivo:** Trovare automaticamente un **modello probabilistico** che descriva la configurazione dei dati. L'obiettivo è raggruppare i dati (*clustering*) o ridurre la dimensionalità, basandosi solo sulle proprietà intrinseche dei pixel (intensità, posizione).
- **Esempi:** **Algoritmi di *clustering*** come il *K-Means* o il *Fuzzy C-Means* (FCM), che raggruppano i pixel in *cluster* in base alla loro intensità, assumendo che i *cluster* rappresentino strutture anatomiche diverse.

Si parla di **apprendimento non supervisionato** (*unsupervised learning*) se nel processo non viene utilizzato un **set di dati validato** (cioè, dati di *training* con etichette predefinite).

Lo scopo principale del processo è trovare un **modello di tipo probabilistico** che descriva la configurazione o la struttura intrinseca dei dati stessi.

Nota: Anche negli algoritmi *unsupervised* si adotta un “**modello**” dei dati (*ipotesi a priori*) ottenuto dalla conoscenza del problema, anche se i dati non sono definiti in modo esplicito o etichettato. Molti algoritmi di **segmentazione** (come gli algoritmi di *clustering*) rientrano in questa classe.

Il Problema dell'Approssimazione di Funzione e Spazio delle Ipotesi

Consideriamo un esempio semplice di approssimazione:

Sia data una funzione incognita $f(x)$. Conosciamo solo alcuni **campioni** di $f(x)$, ovvero una serie di coppie $(x, f(x))$. L'obiettivo è trovare una funzione h che approssimi al meglio possibile f sulla base dei campioni posseduti.

In modo formale, ogni possibile funzione h è una **ipotesi** di come sia f . Il problema è:

1. Verificare quanto l'ipotesi h sia corretta.
2. Cambiarla in modo da ottenere delle h sempre più simili a f .

Il Rasoio di Occam (*Occam's Razor*)

Supponiamo che le coppie $(x, f(x))$ siano numeri reali. Limitiamo lo **spazio delle ipotesi** H a tutti i polinomi di grado massimo k .

La figura (a) mostra tre campioni dove una funzione h lineare ottiene un *fitting* perfetto. Un *fitting* perfetto viene anche ottenuto da una funzione polinomiale di ordine superiore nella figura (b).

- **Problema della Scelta:** Quale delle due ipotesi h dobbiamo preferire?
- **Soluzione (Rasoio di Occam):** Intuitivamente, si preferisce l'ipotesi più semplice. Questa euristica è detta del **rasoio di Occam**:

Preferire l'ipotesi più semplice tra quelle consistenti con i dati.

In questo caso, la semplicità può essere considerata equivalente al **grado del polinomio**.

L'Impatto dello Spazio delle Ipotesi (H)

La possibilità di trovare un'ipotesi h che fitti correttamente i dati dipende fortemente dallo **spazio delle ipotesi** H scelto:

- Se $f(x)$ è una funzione sinusoidale, sarà improbabile trovare una funzione polinomiale che la descriva correttamente all'interno dello spazio $H_{polinomiale}$.
- Invece, utilizzando lo spazio $H_{sinusoidale}$ (delle funzioni sinusoidali), si otterrebbe facilmente un risultato corretto.

In definitiva, l'algoritmo dipenderà sempre da una **ipotesi a priori** sulla natura di $f(x)$. Analogamente, in un algoritmo di *clustering*, il risultato dipenderà dal **numero di cluster**, che deriva anch'esso da un'ipotesi a priori sulla distribuzione dei dati.

Concetti Utili per la Segmentazione basata su ML

Estrazione delle Caratteristiche (*Feature Extraction*) Nei metodi *Machine Learning* tradizionali (non *Deep Learning*), il successo dipende dalla capacità di estrarre **caratteristiche discriminanti** dall'immagine. Queste caratteristiche possono essere:

- **Intensità:** Valori di grigio assoluti o normalizzati.
- **Caratteristiche Locali (Texture):** Varianza locale, entropia, o parametri derivati dagli istogrammi del vicinato del pixel.
- **Caratteristiche di Forma:** Misure relative alla forma dell'oggetto (usate dopo una prima segmentazione).

Vantaggio del *Deep Learning*: Le reti neurali (*Deep Learning*) sono in grado di eseguire la **feature extraction** automaticamente durante l'addestramento, a differenza degli algoritmi ML tradizionali dove le *features* dovevano essere definite manualmente dall'esperto.

***Deep Learning* (Apprendimento Profondo)** Il *Deep Learning* è un sottoinsieme del *Machine Learning* che utilizza **Reti Neurali Artificiali (ANN) con più strati nascosti (deep)**.

- **Architetture Comuni:** Per la segmentazione, l'architettura più usata è la **Convolutional Neural Network (CNN)**.
- **Segmentazione Semantica:** Le CNN sono in grado di eseguire la *segmentazione semantica*, in cui ogni pixel nell'immagine viene classificato in una categoria specifica (es. "fegato", "rene", "sfondo"). Un esempio celebre è l'architettura **U-Net**, molto efficace nelle applicazioni biomediche.

Funzione di Errore (*Loss Function*) Nella segmentazione supervisionata, la *loss function* (funzione di costo) è l'elemento cruciale che guida l'apprendimento, quantificando la distanza tra la predizione del modello e la *Ground Truth*.

- **Loss di Cross-Entropia:** Comune per problemi di classificazione.
- **Loss di Dice:** Particolarmente popolare in ambito medico, poiché misura la sovrapposizione tra la regione segmentata e la regione reale:

$$\mathcal{L}_{Dice} = 1 - \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

dove A è la predizione e B è la *Ground Truth*. Minimizzare questa *loss* massimizza la sovrapposizione volumetrica tra le due maschere.

Misura delle Prestazioni e Curva di Apprendimento

Per verificare la qualità del processo di apprendimento (*learning*), l'algoritmo sviluppato viene confrontato con un **"gold standard"**, cioè un insieme di dati di test (*test set*) in cui la funzione

vera $f(x)$ è nota.

Metodo di Validazione e Misura dell'Errore

Il metodo per misurare le prestazioni, e quindi la capacità di generalizzazione dell'ipotesi H trovata, si schematizza come segue:

1. **Raccolta Dati:** Ottenere un insieme di esempi il più numeroso possibile.
2. **Suddivisione:** Dividere l'insieme totale in due parti: **Training Set** (insieme di addestramento) e **Test Set** (insieme di prova).
3. **Addestramento:** Applicare l'algoritmo di *learning* al **Training Set** per ottenere l'ipotesi (funzione) H .
4. **Verifica:** Applicare H al **Test Set** (dati mai visti prima dall'algoritmo) e verificare l'errore commesso.

Suggerimento: Cross-Validation Per una trattazione completa della **cross-validation** e della **k-fold cross-validation**, vedi Introduzione alla Classificazione dove queste tecniche sono descritte nel contesto del deep learning.

5. **Ripetizioni:** Ripetere i passi precedenti più volte per diverse scelte casuali (*random*) del *Training Set*, variando anche la sua dimensione.

La Learning Curve

La ripetizione del processo (Passo 5) consente di ottenere una funzione che descrive la capacità dell'algoritmo di trovare un'ipotesi H consistente con i dati, in funzione della **grandezza del Training Set**.

Questa funzione rappresenta la cosiddetta **Learning Curve** (curva di apprendimento).

- **Descrizione:** La *Learning Curve* descrive la capacità dell'algoritmo di apprendere da un insieme di dati di addestramento di una certa dimensione.
- **Forma Ideale:** Una buona *Learning Curve* dovrebbe essere **monotona crescente** (indicando che più dati portano a una maggiore precisione e generalizzazione) e viene definita un **“happy graph”**.

Le **Reti Neurali Convolutione (CNN)** utilizzate nella segmentazione utilizzano in generale questo approccio di **apprendimento supervisionato**.

Procedure di Ottimizzazione

L'**ottimizzazione** è la ricerca della combinazione di parametri che **massimizza o minimizza** una certa funzione (la *metrica*).

Il Problema del Commesso Viaggiatore (TSP)

Il **Problema del Commesso Viaggiatore (TSP)** è un esempio classico di problema di ottimizzazione.

- **Definizione:** Dato un insieme di clienti (nodi) e le distanze tra loro (archi), trovare un ciclo che tocchi tutti i nodi **una sola volta** e abbia la **durata complessiva minima**.

Approccio a Ricerca Esaustiva (*Brute Force*) Questo approccio calcola tutte le possibili soluzioni e sceglie l'ottima. Se il dominio di *input* è finito, trova sempre la soluzione corretta.

- **Complessità:** Per n nodi, il numero di soluzioni possibili è $(n-1)!$. Se si considera anche il calcolo della lunghezza del percorso, il numero di operazioni è $N_O = n \cdot (n-1)! = n!$.
- **Esempio di Complessità:** Per $n = 20$ nodi, $20! \approx 2.4 \times 10^{18}$ operazioni. Su un PC da 4 Gflops (4×10^9 op/s), il tempo necessario è di circa **20 anni**.

L'esempio dimostra come la ricerca esaustiva non sia applicabile al crescere della dimensione del problema.

Problemi NP-Completi Il problema TSP è un esempio della famiglia di problemi **NP-completi** (Non-deterministic Polynomial time). Questi sono problemi la cui complessità cresce in modo **non lineare** con la dimensione dell'input e non possono essere risolti in modo esaustivo per dimensioni dei dati superiori a una certa soglia.

Soluzioni Approssimate (Non Esaustive)

Poiché i problemi NP-completi non possono essere risolti in modo esaustivo, si utilizzano algoritmi che ottengono **soluzioni approssimate** con una complessità computazionale accettabile. Tali algoritmi, tuttavia, **non possono fornire una soluzione sicuramente ottima**.

Ricerca Casuale (*Random Search*) La ricerca casuale esplora un percorso di ricerca variabile in modo *random*.

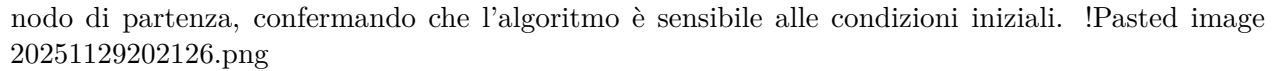
- **Utilità:** Se esplora solo una parte del dominio, ha una probabilità di trovare l'ottimo pari al rapporto tra percorsi esplorati e percorsi totali.
- **Funzione:** Non ha utilità pratica diretta, ma serve come **confronto** per gli altri algoritmi: qualsiasi algoritmo di ottimizzazione ragionevole deve funzionare meglio della ricerca casuale.

Algoritmi Greedy (Approccio *Best-First-Search*) Un algoritmo **Greedy** (avido) è un esempio di soluzione approssimata che opera con l'approccio **best-first-search**: si muove in un grafo scegliendo, passo dopo passo, i nodi che sembrano migliori per risolvere il problema a livello locale.

- **Esempio TSP (Con Successo):** Partendo dal nodo 1, l'algoritmo sceglie sempre il nodo non visitato più vicino. Il percorso (1-2-3-4-5-1) ha lunghezza 36, che in un certo grafo può essere la minima.
- **Complessità:** L'ordine di grandezza del numero di passi è $\mathcal{O}(N^2)$, molto minore di $\mathcal{O}(N!)$ dell'algoritmo esaustivo.
- **Limite:** Non è detto che trovi la soluzione migliore. Ad esempio, in un grafo modificato, l'algoritmo *greedy* può trovare il percorso 36, mentre ne esiste uno migliore con lunghezza 35 (1-5-2-3-4-1), che viene "saltato" perché la scelta locale non è ottimale a livello globale.

Risultato: L'algoritmo *greedy* fornisce una soluzione approssimata di limitata complessità computazionale, ma non necessariamente la migliore.

- **Dipendenza dalle Condizioni Iniziali:** La soluzione prodotta dall'algoritmo *best-first-search* dipende dal **nodo iniziale** che si sceglie.

La figura (Berlin52) mostra che soluzioni diverse (es. migliore 8182) si ottengono variando il nodo di partenza, confermando che l'algoritmo è sensibile alle condizioni iniziali. 

Caratteristiche di un Processo di Ottimizzazione

In generale, un processo di ottimizzazione è definito da tre caratteristiche fondamentali:

1. **Spazio di Ricerca (*Search-space*):** L'insieme dei valori che possono assumere le possibili soluzioni (es. tutte le possibili sequenze di nodi senza ripetizioni nel TSP).
2. **Metrica (o Funzione Obiettivo):** La quantità da massimizzare o minimizzare (es. la lunghezza di un percorso nel TSP).
3. **Processo di Ottimizzazione (Algoritmo):** L'algoritmo utilizzato per trovare all'interno dello *search-space* la soluzione che ottimizza la metrica (es. il metodo *greedy*).

Classificazione degli Algoritmi di Ottimizzazione

Gli algoritmi di ottimizzazione si classificano in base alla dipendenza dalle condizioni iniziali:

Categoria	Caratteristiche	Esempio TSP
Ottimizzatori Locali	Trovano una soluzione partendo da un punto specifico dello <i>search-space</i> (condizioni iniziali). La soluzione dipende dalle condizioni iniziali.	L'algoritmo <i>greedy</i> applicato a un singolo nodo iniziale.
Ottimizzatori Globali	Trovano la soluzione migliore all'interno dello <i>search-space</i> indipendentemente dai parametri di ingresso.	L'unico vero ottimizzatore globale è la ricerca esaustiva .
Approssimazione Globale	Un ottimizzatore locale iterato su tutte le possibili condizioni iniziali (es. <i>greedy</i> su tutti i nodi di partenza) si considera un'approssimazione di un ottimizzatore globale.	<i>Greedy</i> iterato su tutti i nodi.

In Sintesi: Per i problemi NP-completi, è necessario utilizzare un algoritmo di ottimizzazione più veloce dell'esaustivo. Tale algoritmo non garantirà l'ottimo globale, ma solo una soluzione **“ragionevolmente” buona**, che spesso dipenderà dalle condizioni iniziali.